## 前置知識與目標

- 前置知識為字典 ADT、模算術、期望值、鏈結串列及攤銷分析。
- 能區分雜湊函數、碰撞解決與擴容政策，對每項效能結論明訂假設。
- 能追蹤 chaining、linear probing、刪除 tombstone 與重新雜湊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Dictionary ADT 與碰撞

- 字典把每個唯一 key 映到一個 value，操作包含 find、insert-or-update、erase。
- 雜湊函數 `h:U→{0,…,m-1}` 把鍵域壓縮到 m 個桶。若 `|U|>m`，碰撞不可避免。
- key 相等時必須得到相同 hash。反向不成立，hash 相同的不同鍵仍需用 equality 分辨。
- 哈希值不是唯一識別碼。只比 hash 不比鍵，會使碰撞變成資料錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 雜湊函數與機率空間

- 整數 division method 為 $h(k)=k \bmod m$。鍵若具有模 m 的規律，可能集中在同一桶。
- 隨機選函數與假設輸入均勻是不同模型。universal family 要求對任意不同 x、y，隨機 h 的碰撞機率至多 $1/m$。
- 若輸入在選 h 前固定，通用雜湊能以內部亂數界定碰撞期望；適應性攻擊者可能需要更強假設。
- 密碼學雜湊和一般雜湊表服務不同威脅模型。不能只看輸出位數便宣稱抗攻擊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Separate chaining 與負載率

- 每個桶保留同 hash 鍵的串列或其他集合，負載率 $\alpha=n/m$ 可以大於 1。
- 均勻雜湊下，獨立的不成功查詢之預期桶長為 α。若只採通用碰撞上界，且查詢鍵與儲存鍵集合在選函數前固定，桶長期望至多 α，不一定恰等於 α；兩者都給出搜尋期望 $O(1+\alpha)$，前提是鍵操作為常數時間。
- 成功搜尋的期望還與插入次序及查詢分布有關，但在常見均勻模型下也為 $O(1+\alpha)$。
- 最壞仍可有 n 個鍵同桶，搜尋為 $\Theta(n)$。隨機化給期望保證，沒有把最壞情況消掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chaining 例子

- 設 m=7、`h(k)=k mod 7`，依序插入 `10,17,24,8`。
- 桶 3 依插入尾端的政策保存 `[10,17,24]`，桶 1 保存 `[8]`，其餘空。
- 查 24 若用串列線性掃描，需檢查三個鍵。查 31 也會掃過桶 3，不能因 n/m 小便宣稱本次必為常數。
- 若需頻繁保留排序或處理攻擊性碰撞，可改用樹化桶。這會改變實作和成本，不屬於基本 chaining 自動具有的能力。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Chaining 的完整字典操作

```
find(k):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): return reference(node.value)
    return absent
insert_or_update(k,v):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): node.value=v; return updated
    prepend newly constructed pair(k,v) to bucket[j]; n+=1
erase(k):
    j=hash(k) mod m; predecessor=dummy_before(bucket[j])
    while predecessor.next!=null:
        if equal(predecessor.next.key,k): unlink and release; n-=1; return true
        predecessor=predecessor.next
    return false
```

本程式採頭插，與先前示例的尾插政策不同：10,17,24都入桶3後，頭插順序為[24,17,10]；find24一次比較，find31三次。刪17後為[24,10]；再次insert24只更新value，不增加n。兩種插入政策都實作同 Dictionary ADT，但成功搜尋的個別成本不同，不能拿一張trace當所有政策共同結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Chaining 的完整字典操作 （續）

dummy_before是操作前驅，不必是常駐資料節點；刪首桶節點時要接回bucket head。平均鏈長α由鍵與hash模型決定；碰撞集中時最壞Θ(n)。插入若不查重而直接頭插，會變成允許重複的multimap而不是本章唯一key的dictionary。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 全表碰撞對與固定查詢的不同分母

```
// S is fixed before the random function h is selected.
tau=0
for each unordered pair {x,y} in S:
    tau += indicator(h(x)==h(y))
E[tau] = sum_{unordered {x,y}} Pr(h(x)==h(y))
```

一次不成功查詢的鏈長只把固定x與n個儲存鍵比較，期望上界n/m。全表碰撞對則有binom(n,2)個不同配對：若family為universal，即每對不同鍵碰撞機率≤1/m，便有E[τ]≤n(n−1)/(2m)。若每對恰1/m，則為等號。兩者都只需線性期望，不要求所有配對事件獨立；但不能把n/m誤當整張表的總碰撞數。

例如m=Θ(n^(4/3))時，全表E[τ]=O(n^(2/3))；固定查詢鏈長E[X]≤n/m=O(n^(−1/3))，還須加至少一次桶定位成本1。若m=Θ(n)，總碰撞期望可為Θ(n)，仍與單次查詢期望O(1)相容。碰撞對數與非空桶數也不同：單桶有b鍵就貢獻b(b−1)/2對，不是b−1對。

「最壞期望」在此指 $\max_{|S|=n} \mathbb{E}_h[\tau(S,h)]$。先固定任意 $S$，再對 $h$ 取期望，通用界對每個 $S$ 同時有效。它不是 $\mathbb{E}_h[\max_{|S|=n} \tau(S,h)]$：後者容許看過 $h$ 才挑輸入，量詞次序改變。對非負 $\tau$ 及 $a>0$，還可用 Markov 得到 $\Pr(\tau \geq a) \leq \mathbb{E}[\tau]/a$；小期望不表示每次選函數都沒有碰撞。

## 通用雜湊族的構造與偽隨機化反例

```
// p prime, keys are distinct vectors in F_p^2.
choose a,b independently and uniformly from 0..p-1
h(a,b,(x1,x2))=(a*x1+b*x2) mod p
// A random output translation of fixed g:
g_c(x)=(g(x)+c) mod m
```

對不同向量x,y，令δ=(x1−y1,x2−y2)≠(0,0)。若δ1≠0，固定任意b後，aδ1+bδ2=0在有限域中恰有一個a，因δ1有乘法反元素；故碰撞機率恰1/p。若δ1=0，則δ2≠0，同理由b解出唯一值。此證明包含a=b=0的函數，不能因它單次全碰撞就否定整個family的期望保證。

p=11，固定四鍵(0,0),(1,0),(0,1),(1,1)，共有6對；每對在121種(a,b)中恰11種碰撞，所以全表期望6/11。函數a=b=0使6對全碰撞；這是模型允許的低機率事件，期望仍按全部121函數計。

相反地，任意固定g再隨機選c平移輸出，g_c(x)=g_c(y)若且唯若g(x)=g(y)，原碰撞完全保留。例如g(x1,x2)=x1，鍵(3,1),(3,7)永遠碰撞，故不是通用族。把大整數先對p取模也可能先把不同鍵合成相同表示；若鍵域為0..$p^2$−1，應以兩位base-p向量作一對一編碼，而非只保留最低位。



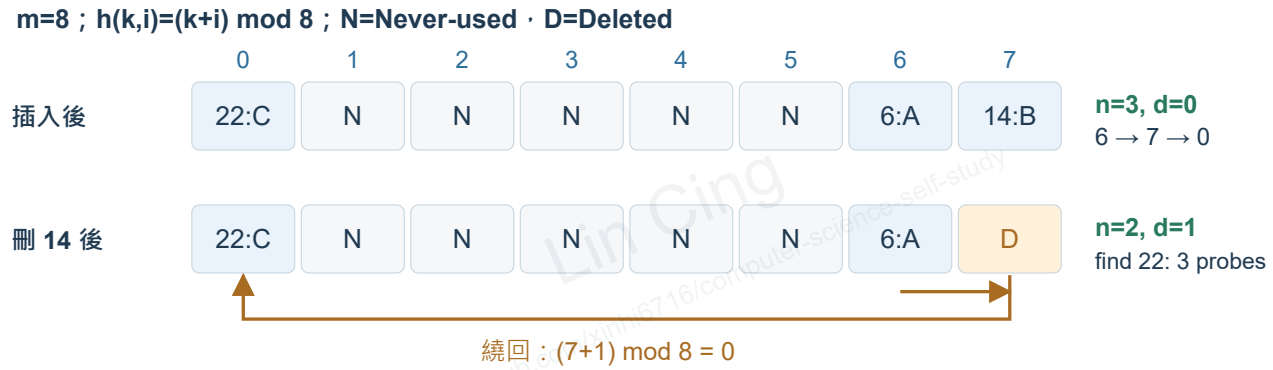
## Open addressing 的槽位狀態

- 所有元素放在表內，每鍵依探測序列 `h(k,0),h(k,1),…` 尋找位置。通常要求 `n<m`。
- 槽位分為 Never-used、Occupied、Deleted，三種狀態語義不同。
- 搜尋遇到 Never-used 可停止，因插入時不會越過它卻把同鍵放後面。遇到 Deleted 必須繼續。
- 插入可記住首個 Deleted，但仍要繼續檢查後面是否已有同鍵，避免插入重複 key。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Linear probing：跨尾端的碰撞鏈

固定 m=8，$h(k,i) = (k+i) \bmod 8$。依序插入 (6,A)、(14,B)、(22,C)，三鍵的 home 都是槽 6。n 為活鍵數，d 為 Deleted 槽數；N 表示本次建表後未使用的 Never-used。




刪 14 只把槽 7 變 D。找 22 仍檢查槽 6、7、0，共 3 次；若誤把槽 7 變 N，就在第二次探測錯判不存在。D 不保留可讀的舊 pair，只保留「搜尋不能在此停下」的資訊。



## Primary clustering：不同 home 也會延長同一段

另從刪除前的三鍵狀態開始，依序插入 8、1。下表只列鍵，N 表示尚未使用；兩次都不需擴容。

| 動作 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 初始 | 22 | N | N | N | N | N | 6 | 14 |
| 插 8：探 0→1 | 22 | 8 | N | N | N | N | 6 | 14 |
| 插 1：探 1→2 | 22 | 8 | 1 | N | N | N | 6 | 14 |

8 的 home 是 0，1 的 home 是 1，都不是原三鍵的 home=6，卻仍延長跨尾端的連續區 6→7→0→1→2。之後 home 落在這段內的新鍵，也必須掃到區段尾端以外。這就是 primary clustering：影響不只限於 home 完全相同的鍵。連續探測有局部性，但其機率行為不同於獨立的均勻隨機排列。

## 同一個 Deleted，更新與新增要做不同事

從刪 14 後的狀態開始，先 `insert_or_update(22,C*)`，再 `insert_or_update(30,Z)`。

**更新 22 → C***

① 槽 6：key=6，不相等

② 槽 7：Deleted；記住候選 7

③ 槽 0：找到 key=22

更新槽 0；不占用候選槽 7

**n=2，d=1；仍只有一筆 22**

**新增 30 → Z**

① 槽 6：key=6，不相等

② 槽 7：Deleted；記住候選 7

③ 槽 0：key=22，不相等

④ 槽 1：Never-used，確認不存在

**回寫槽 7；n=3，d=0**

第一次在槽 0 更新 22，不使用候選槽 7。第二次探到槽 1 的 N 才確定 30 不存在，然後回寫最早的 D 槽 7。最後槽 0、6、7 分別為 22:C*、6:A、30:Z。



## Never-used 的停止理由，Deleted 的繼續理由

不變量是：每個現存鍵從自己的 home 探到儲存位置之前，都不會先遇到 N。插入以第一個 N 作為不存在的證據；刪除 O→D 不製造新的 N；重用 D 也不改變其他鍵前方的 N。因此此不變量在這三種操作後仍成立。

| 錯誤改動 | 探測或寫入結果 | 破壞的契約 |
|---|---|---|
| 刪 14 後將槽 7 設 N | 找 22 只查 6→7 就停止，漏掉槽 0 | 現存鍵的可達性 |
| 更新 22 時見 D 就立即寫槽 7 | 槽 7 與槽 0 都有 key=22 | Dictionary 的鍵唯一性 |

`insert_new` 若要求呼叫端已證明鍵不存在，可省略查重；本章的 `insert_or_update` 沒有這項前提，必須找到同鍵、遇 N 或查滿 m 槽才作決定。演算法第 11 章把相同的停止狀態稱為 EMPTY；它與這裡的 Never-used 對應，兩科都不把 Deleted 當停止點。

Backshift deletion 是另一種刪除策略，必須按 home 與探測區間合法搬移，維持每個現存鍵的可達性；它不是把 tombstone 原地改成 N。

## 沒有 Never-used，仍要在一輪後結束

先單獨檢查固定容量的 `locate_or_slot`，暫停自動擴容與清理。依序插 6、14、22、1、2、3、4、5，填滿 8 槽；再刪除除 22 外的七鍵。此時 n=1、d=7，沒有 N。

查不存在的 38 時，home=6；依序檢查 6、7、0、1、2、3、4、5，記住第一個 D=6。八槽都沒有 38，便回傳 absent(6)，不能一直等待 N。若八槽全為 Occupied 且沒有同鍵，則回傳 full。

這個邊界狀態用來驗證探測程序。後文的自動維護政策會較早清理，不會讓表長期留在 d=7。**固定容量的合法狀態，不等於該維護政策容許保留的狀態。**



## 同容量 rehash：重新定位，而非只清標記

沿用只剩 22 的固定容量狀態；新建同樣 8 槽的 N 表，只把 live pair 按新表的探測序列重插。




查 38：前 6→7→0→1→2→3→4→5（8 次）；後 6→7（2 次）

22 由槽 0 搬到槽 6，n 仍為 1，d 歸零。現在查 38 只探 6、7。若僅把舊 D 改 N 而保留 22 在槽 0，從 home=6 就會立刻錯停。清理減少了搜尋必須跨過的標記，但也花費掃表與重插入的成本；後文的清理門檻會把這些工作一起計帳。

## Double hashing 與探測覆蓋

- $h(k,i)=(h_1(k)+i\cdot h_2(k)) \bmod m$。步長要與 m 互質，才會遍歷全部 m 個槽。
- 原例 m=8、步長 2，只能訪問同奇偶的 4 個槽，可能在表仍有空位時插入失敗。
- 選 m 為質數且令 $1\le h_2(k)<m$，可保證步長互質；也可用 2 的冪容量配奇數步長。
- 二次探測也需要指定係數、表大小與負載限制，不能只寫 $h+i^2$ 就保證找到所有空位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 查找或候選槽：先回傳判定，不立即寫入

```
locate_or_slot(key):
    first_deleted=none
    for i=0..m-1:
        j=probe(key,i)
        if state[j]==Occupied and equal(keys[j],key): return found(j)
        if state[j]==Deleted and first_deleted==none: first_deleted=j
        if state[j]==Never_used:
            return absent(first_deleted if first_deleted!=none else j)
    if first_deleted is set: return absent(first_deleted)
    return full
```

此程序最多 m 次探測，要求 probe 序列覆蓋全表。只有 Occupied 才讀取 keys[j]；Deleted 沒有 live pair，不得解參照已銷毀的鍵。回傳 found、absent(slot) 或 full，讓呼叫端分開處理更新、插入與容量不足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 自動插入與刪除：重建後必須重找位置

```
insert_or_update(k,v):
    result=locate_or_slot(k)
    if result is found(j): values[j]=v; return
    if n+1>3*m/4: rehash(2*m); result=locate_or_slot(k)
    else if deleted>m/8: rehash(m); result=locate_or_slot(k)
    j=result.slot
    if state[j]==Deleted: deleted-=1
    construct pair(k,v) at j; state[j]=Occupied; n+=1

erase(k):
    if locate_or_slot(k) is found(j):
        destroy item; state[j]=Deleted; n-=1; deleted+=1
        if m>minimum_capacity and n<m/4:
            rehash(max(minimum_capacity,m//2))
```

先處理同鍵更新，再測擴容與清理門檻。rehash 可能改變位置，所以之後重新呼叫 locate_or_slot；不能把舊槽索引帶進新表。以上 8 槽的更新／新增例 d=1≤m/8、live 負載也未超門檻，不會額外重建。原 7 槽練習則明定固定容量、不套這層政策。



## 探測期望公式的適用模型

- 理想 uniform hashing 假設每鍵的探測排列均勻，且獨立於先前建表歷史。沒有 tombstone 且 `α<1` 時，不成功搜尋期望至多 `1/(1-α)`。
- 同模型，鍵插入後未刪除，且成功查詢等機率選取現存鍵時，`α>0` 的期望可界為 `(1/α) ln(1/(1-α))`。
- α=0.9 時，前一個界為 10，而非不隨負載改變的神奇常數。α 趨近 1，成本界發散。
- 此頁不是 linear probing 的專屬精確公式，也未包含 tombstone、hash 計算及長字串相等比較成本。

## 理想探測上界的推導

- 假設尚有 m−n 個 Never-used 槽、沒有 tombstone，且每鍵探測序列是獨立於已佔槽集合的均勻隨機排列。前 i 次都命中已占槽的機率為 $(n)_i/(m)_i \le \alpha^i$，其中 $(n)_i$ 是下降階乘。
- 以尾和公式，探測數 X 的期望 $E[X]=\Sigma_{i\ge 0}Pr(X>i)\le \Sigma \alpha^i=1/(1-\alpha)$。沒有 tombstone 與 α<1 都參與這個推導。
- 成功查詢等機率選 n 個插入且未刪的鍵，對插入時負載 j/m 取平均。因 $1/(1-x)$ 遞增，左矩形和小於積分，故 $E[X]\le(1/n)\Sigma_{j=0}^{n-1}1/(1-j/m)\le(1/\alpha)\ln(1/(1-\alpha))$。
- 此證明展示理想機率模型如何進入數學式。對線性探測的連續 cluster，均勻隨機排列前提不成立。



## 擴容與攤銷不能省略重算 hash

- 表大小改變時，hash 到槽位的映射也改變，須把現存鍵重新插入。不能只把原陣列複製到較大空間。
- 負載率達固定門檻時倍增，當 m=Θ(n)，掃描與配置為 Θ(n)；重新插入在適當隨機假設、負載遠離 1 時為期望 Θ(n)。任意碰撞下不能一概保證 rehash 最壞線性。這些條件下，連續插入才有期望攤銷 $O(1)$ 成本。
- 這同時使用機率假設與跨序列計帳，應明寫「期望攤銷」，不能只稱最壞常數。
- 若縮容門檻與擴容門檻相同，邊界附近交替插入刪除可能反覆重建。不同門檻能避免此震盪。

## 重新雜湊的鍵重定位與期望分析

```
rehash(new_m):
    allocate new_m Never_used slots
    choose compatible new hash parameters if required
    for each old Occupied pair (k,v):
        insert into new table using its new probe sequence
    commit new table; deleted=0; preserve n; release old table
```

以m=7的鍵10,17,24,8為例，h=k mod7時桶3=[10,17,24]、桶1=[8]。改成m=14、h=k mod14，新桶10=[10,24]、桶3=[17]、桶8=[8]；不能把原桶位置保留。開放定址同樣要按新probe重新插入，Deleted全數清除。

以固定不成功查詢鍵 $x \notin S$ 與固定儲存集合 $S$、$|S| = n$ 作通用雜湊分析：$X = \sum_{y \in S} \mathbf{1}[h(y) = h(x)]$，$\mathbb{E}[X] = \sum_{y \in S} \Pr(h(y) = h(x)) \le n/m$。這裡只需線性期望，不需各碰撞事件互相獨立；但 $x, S$ 需在選函數前固定，適應性攻擊者不是同一模型。若 $x \in S$，自碰撞項必為 $1$，整個桶長期望的相應上界是 $1 + (n-1)/m$，不能沿用不成功查詢的界。成功查詢若分布集中在最長鏈，也不能把均勻查詢平均當成該分布保證。



## 重新雜湊的鍵重定位與期望分析 （續）

重建掃描Θ(m+n)；m=Θ(n)且新表負載有固定上限與適當隨機模型時，重插入期望Θ(n)。若hash全碰撞且insert必查重，重插入可能二次；「rehash最壞線性」不可無條件使用。維持擴容3/4、縮容1/4並清理tombstone，才能結合期望與攤銷而得更新序列成本。

## 清理門檻的反例與攤銷記帳

```
// m starts at a power-of-two minimum_capacity>=8.
// Before inserting a genuinely absent key:
if n+1>3*m/4: rehash(2*m)
else if deleted>m/8: rehash(m)
// After deleting a present key:
if m>minimum_capacity and n<m/4:
    rehash(max(minimum_capacity,m//2))
// Rehash resets deleted=0 and reinserts all n live entries.
```

擴縮容量與清 tombstone 是兩種不同的觸發原因。錯法「n+deleted+1>3m/4就同容量重建」會在 n=3m/4 時出問題：刪一鍵後 n=3m/4−1、deleted=1，再插回原鍵，雖可重用那個 Deleted 槽，錯門檻卻立即重建。m=128、96鍵反覆刪95再插95，共500對操作，舊規則重建500次、重插47,500筆，不能用常數個操作支付每次Θ(m)工作。



## 清理門檻的反例與攤銷記帳 （續）

修正後只在 deleted>m/8 時做同容量清理；每次至少累積Ω(m)次刪除，才能支付重建。插入時先查同鍵，更新既有value不觸發容量變化；真正新鍵先測live負載3/4，再測deleted門檻。每次插入完成，n+deleted≤7m/8：若不重建，n≤3m/4且deleted≤m/8；若重建，deleted歸零；重用Deleted不增加已使用槽數。刪除只把Occupied換Deleted，總使用數不變，故任意後續查詢仍至少有m/8個Never-used槽。

容量倍增後live負載約3/8，縮半後至多1/2；下一次相反方向的容量調整需要Ω(m)次live增減，不會一刪一插震盪。同容量清理另由其累積刪除數記帳。各重建在明示的均勻探測／相容亂數模型下為期望Θ(m+n)，可分別攤到觸發操作；因此總更新有期望攤銷O(1)探測成本，最壞碰撞仍可線性或使重插二次。雙重雜湊在2冪容量需奇數步長；此條件只保證遍歷，不等於理想隨機排列假設。

## 溢位處理與工程條件

- chaining 的 overflow 是桶內元素增加；open addressing 的滿表必須偵測，不能無限探測。
- 若 key 是長度 L 的字串，第一次計算 hash 至少可能需 $\Theta(L)$。只把槽位探測數當總時間會漏掉鍵成本。
- 若查詢者能控制大量碰撞鍵，均勻分布假設可能不成立。可用隨機 seed、合適的 keyed hashing 或不同容器。
- 若需求含排序、區間查詢或前驅查詢，平衡搜尋樹通常更直接，因雜湊表不保留鍵次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考文獻與延伸閱讀

- 假設與期望成本核對 MIT 6.006 2008 L5–L7。
- C++ 無序容器的平均與最壞成本、hash/equality 要求，核對 unordered associative requirements。不能將標準平均保證說成每次最壞常數。
- MIT 6.006 2014 Recitation 10, § 1.1：Deleted 的搜尋語意及重複鍵更新。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study